Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales

Sistemas de Computación

TRABAJO PRÁCTICO N.º 4

Módulos del kernel de Linux

Compilación cruzada de un módulo para Raspberry Pi 5

GRUPO

dontReadME

INTEGRANTES

Valentino Di Paolo

Maria Pia Acuña

Córdoba, noviembre de 2026

1. Consigna

> **¿Qué es exactamente un módulo del núcleo?** Los módulos son fragmentos de código que se pueden cargar y descargar en el kernel según se requiera. Extienden la funcionalidad del kernel sin necesidad de reiniciar el sistema. Por ejemplo, un tipo de módulo es el controlador de dispositivo, que permite que el núcleo acceda al hardware conectado al sistema. Sin módulos, tendríamos que construir kernels monolíticos y agregar nuevas funciones directamente en la imagen del kernel. Además de tener kernels más grandes, esto tiene la desventaja de requerir que reconstruyamos y reiniciemos el kernel cada vez que queramos una nueva funcionalidad.
>
> Croscompilar el módulo desde la PC para ARM, intentar cargar el módulo en la PC y verificar que no se puede cargar. Copiarlo en una Raspberry y verificar que se puede cargar.

2. Marco teórico

2.1 Módulos del kernel

Un módulo del kernel (*Loadable Kernel Module*, LKM) es un archivo objeto ELF con extensión `.ko` que se enlaza dinámicamente con el kernel en ejecución. A diferencia de un programa de usuario, no tiene una función `main()`: cuando se carga con `insmod`, el kernel ejecuta su función de inicialización (registrada con `module_init`), y cuando se descarga con `rmmod`, ejecuta su función de salida (registrada con `module_exit`). El código del módulo corre en espacio de kernel y solo puede usar las funciones que el propio kernel exporta, como `printk`; no tiene acceso a la biblioteca estándar de C.

2.2 Compilación cruzada

Compilar de forma cruzada es generar código para una arquitectura distinta de la de la máquina que compila. En este trabajo, el *host* es una PC x86_64 y el *target* es una Raspberry Pi 5, con CPU ARM64 (aarch64). Se usa una cadena de herramientas cruzada (en Ubuntu, el paquete `crossbuild-essential-arm64`, que provee `aarch64-linux-gnu-gcc`), y se le indica al sistema de compilación del kernel (*kbuild*) la arquitectura destino con las variables `ARCH=arm64` y `CROSS_COMPILE=aarch64-linux-gnu-`.

Durante la compilación del kernel conviven dos compiladores. El compilador cruzado genera el código del kernel y de los módulos (líneas `CC` y `CC [M]` de la salida de `make`). El compilador nativo de la PC genera herramientas auxiliares que se ejecutan en el host durante la propia compilación (líneas `HOSTCC`, por ejemplo `lib/gen_crc32table`).

2.3 Por qué un módulo depende del kernel para el que se compiló

Al ejecutar `insmod`, el kernel valida el módulo antes de ejecutar su código. Según la implementación del cargador de módulos del kernel 6.12 (`kernel/module/main.c` y `kernel/module/version.c`), las tres verificaciones relevantes para este trabajo son:

1. **Arquitectura del ELF.** El campo `e_machine` del encabezado ELF tiene que coincidir con la arquitectura del kernel: 62 corresponde a `EM_X86_64` y 183 a `EM_AARCH64`. Si no coincide, el módulo se rechaza con el mensaje `Invalid architecture in ELF header`.

2. **`vermagic`.** Es una cadena grabada en el módulo con la versión del kernel y sus opciones de compilación, por ejemplo `6.12.114-v8-16k-tp4+ SMP preempt mod_unload modversions aarch64`. Cuando el módulo incluye sumas de verificación de símbolos (punto 3), el kernel ignora la parte de la versión y compara solo las opciones.
3. **Sumas de verificación de símbolos (`CONFIG_MODVERSIONS`).** Por cada función del kernel que usa el módulo (por ejemplo `_printk`) se graba un CRC calculado a partir de su declaración y de los tipos que involucra, y el kernel lo compara con el suyo. El módulo obtiene esos CRC del archivo `Module.symvers`, que se genera recién al compilar el kernel completo.

En consecuencia, para compilar un módulo de forma cruzada hace falta el árbol de fuentes del kernel *del target* ya compilado, con su `Module.symvers`. Además, para que el módulo cargue, el kernel que corre en el target tiene que tener los mismos CRC. Para garantizarlo, compilamos el kernel de la Raspberry Pi en la PC, lo instalamos en la Raspberry Pi y compilamos el módulo contra ese mismo árbol de fuentes.

3. Entorno de trabajo

	Host (compilación)	Target (ejecución)
Equipo	PC `tino-Smart-Pro-Q7-i7`	Raspberry Pi 5 (`raspberrypi`)
Arquitectura	x86_64	aarch64 (ARM64)
Sistema operativo	Ubuntu	Raspberry Pi OS (Debian GNU/Linux)
Kernel	`7.0.0-38-generic`	original: `6.12.47+rpt-rpi-2712` compilado en este TP: `6.12.114-v8-16k-tp4+`
Compiladores	`gcc` nativo y `aarch64-linux-gnu-gcc` 15.2.0	—

- **Fuente del kernel de la Raspberry Pi:** https://github.com/raspberrypi/linux, rama `rpi-6.12.y`. El commit exacto queda registrado en `rpi-linux-commit.txt`.
- **Módulo de prueba:** `mimodulo`, del repositorio de la cátedra (`part1/module`).

4. Procedimiento y resultados

Los comandos de cada paso también están automatizados en la carpeta `scripts/` (ver la sección 7).

4.1 Punto de partida: el módulo en su arquitectura nativa

Antes de la compilación cruzada, verificamos que `mimodulo`, compilado normalmente para la PC, se carga y se descarga sin errores.

```
cd part1/module
make
mokutil --sb-state
modinfo mimodulo.ko
sudo insmod mimodulo.ko
```

```
sudo dmesg | tail -5
lsmod | grep mimodulo
grep mimodulo /proc/modules
sudo rmmod mimodulo && sudo dmesg | tail -3
```

```
tino@tino-Smart-Pro-Q7-i7:~/Documentos/linux_kernel/kenel-modules/part1/module$ mokutil --sb-state
SecureBoot disabled
tino@tino-Smart-Pro-Q7-i7:~/Documentos/linux_kernel/kenel-modules/part1/module$ modinfo mimodulo.ko
filename:       /home/tino/Documentos/linux_kernel/kenel-modules/part1/module/mimodulo.ko
author:         Catedra de SdeC
description:    Primer modulo ejemplo
license:        GPL
srcversion:     C6390D617B2101FB1B600A9
depends:
name:           mimodulo
retpoline:      Y
vermagic:       7.0.0-38-generic SMP preempt mod_unload modversions
tino@tino-Smart-Pro-Q7-i7:~/Documentos/linux_kernel/kenel-modules/part1/module$ sudo insmod mimodulo.ko
tino@tino-Smart-Pro-Q7-i7:~/Documentos/linux_kernel/kenel-modules/part1/module$ sudo dmesg | tail -5
[  980.870708] Modulo cargado en el kernel.
[ 1012.494566] Modulo descargado del kernel.
[ 1158.358629] Modulo cargado en el kernel.
[ 1197.451195] Modulo descargado del kernel.
[ 1412.452650] Modulo cargado en el kernel.
tino@tino-Smart-Pro-Q7-i7:~/Documentos/linux_kernel/kenel-modules/part1/module$ lsmod | grep mimodulo
mimodulo               12288  0
tino@tino-Smart-Pro-Q7-i7:~/Documentos/linux_kernel/kenel-modules/part1/module$ grep mimodulo /proc/modules
mimodulo 12288 0 - Live 0x0000000000000000 (OE)
tino@tino-Smart-Pro-Q7-i7:~/Documentos/linux_kernel/kenel-modules/part1/module$ sudo rmmod mimodulo && sudo dmesg
| tail -3
[ 1197.451195] Modulo descargado del kernel.
[ 1412.452650] Modulo cargado en el kernel.
[ 1438.882546] Modulo descargado del kernel.
tino@tino-Smart-Pro-Q7-i7:~/Documentos/linux_kernel/kenel-modules/part1/module$
```

Figura 1 — mimodulo compilado para x86_64, cargado y descargado en la PC.

Secure Boot está deshabilitado, así que el kernel acepta el módulo aunque no esté firmado. En `/proc/modules` lo marca como `(OE)`: **O** indica que es un módulo externo al árbol del kernel (*out-of-tree*) y **E**, que no tiene firma. El `vermagic` (`7.0.0-38-generic SMP preempt mod_unload modversions`) corresponde al kernel de la PC.

Estado inicial de la Raspberry Pi:

```
valendipaolo@raspberrypi:~ $ uname -rm
6.12.47+rpt-rpi-2712 aarch64
```

4.2 Paso 1 — Compilación cruzada del kernel de la Raspberry Pi

```
sudo apt install -y git bc bison flex libssl-dev libncurses-dev make crossbuild-essential-arm64
mkdir -p ~/rpi && cd ~/rpi
git clone --depth=1 --branch rpi-6.12.y https://github.com/raspberrypi/linux
cd linux
export ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
make bcm2712_defconfig
./scripts/config --set-str LOCALVERSION "-v8-16k-tp4"
make olddefconfig
make -j$(nproc) Image modules dtbs
```

Decisiones tomadas:

- **Rama `rpi-6.12.y`:** es la misma serie (6.12) que el kernel original de la Raspberry Pi. Así se minimizan las diferencias con el firmware y los device trees que ya estaban instalados.

- **`bcm2712_defconfig`:** es la configuración oficial para la Raspberry Pi 5 (SoC BCM2712), que usa páginas de memoria de 16 KiB.
- **`LOCALVERSION "-v8-16k-tp4"`:** conserva el sufijo propio de esa configuración y le agrega `-tp4`. Así, los módulos del kernel nuevo se instalan en un directorio distinto de los del kernel original, y el kernel se distingue fácilmente en `uname -r`.

La compilación tardó alrededor de 30 minutos. Para documentar el resultado:

```
uname -m
${CROSS_COMPILE}gcc --version | head -1
grep -E '^CONFIG_(LOCALVERSION|ARM64_16K_PAGES|MODVERSIONS)=' .config
make -s kernelrelease
ls -lh arch/arm64/boot/Image; file arch/arm64/boot/Image
find . -name '*.ko' | wc -l; file drivers/media/usb/uvc/uvcvideo.ko
ls -lh Module.symvers; grep -wE '_printk|module_layout' Module.symvers
```

```
== Máquina donde compilé ==
x86_64
== Compilador cruzado ==
aarch64-linux-gnu-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
== Config del kernel (Pi 5) ==
CONFIG_LOCALVERSION="-v8-16k-tp4"
CONFIG_ARM64_16K_PAGES=y
CONFIG_MODVERSIONS=y
== Versión del kernel compilado ==
6.12.114-v8-16k-tp4+
== Imagen del kernel ==
-rw-rw-r-- 1 tino tino 24M Nov  2 15:44 arch/arm64/boot/Image
arch/arm64/boot/Image: Linux kernel ARM64 boot executable Image, little-endian, 16K pages
== Módulos ==
1910 de 1910
drivers/media/usb/uvc/uvcvideo.ko: ELF 64-bit LSB relocatable, ARM aarch64
== Module.symvers ==
-rw-rw-r-- 1 tino tino 1,2M Nov  2 15:44 Module.symvers
0x92997ed8      _printk vmlinux EXPORT_SYMBOL
0xd906d5fe      module_layout   vmlinux EXPORT_SYMBOL
```

Figura 2 — Kernel de la Raspberry Pi 5 compilado en la PC x86_64.

El kernel se compiló en una máquina x86_64 con `aarch64-linux-gnu-gcc` 15.2.0. El resultado es una imagen `Image` para ARM64 de 24 MB, con páginas de 16K, y 1910 módulos también para ARM aarch64. `Module.symvers` contiene el CRC de cada símbolo que exporta el kernel; por ejemplo, el de `_printk` es `0x92997ed8` y el de `module_layout` es `0xd906d5fe`.

4.3 Paso 2 — Compilación cruzada de mimodulo

```
cd part1/module
make clean
make -C $HOME/rpi/linux ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- M=$PWD modules
mkdir -p ~/tp4-arm && cp mimodulo.ko ~/tp4-arm/mimodulo_arm64.ko
file ~/tp4-arm/mimodulo_arm64.ko
modinfo ~/tp4-arm/mimodulo_arm64.ko | grep vermagic
modprobe --dump-modversions ~/tp4-arm/mimodulo_arm64.ko | grep -E '_printk|module_layout'
```

`-C` apunta al árbol del kernel de la Raspberry Pi, no a los headers de la PC, y `M=` apunta al directorio del módulo. No hizo falta modificar el Makefile de la cátedra, porque kbuild solo lee de él la línea `obj-m`.



Figura 3 — Compilación cruzada de mimodulo y verificación del binario.

- `file` muestra que el módulo es un `ELF 64-bit LSB relocatable, ARM aarch64`.
- El `vermagic` es `6.12.114-v8-16k-tp4+ SMP preempt mod_unload modversions aarch64`.
- Los CRC de `_printk` y `module_layout` que quedaron grabados en el módulo son idénticos a los de `Module.symvers` (Figura 2): el módulo quedó ligado a ese kernel.

Las dos advertencias `-Wmissing-prototypes` aparecen porque `modulo_lin_init` y `modulo_lin_clean` no están declaradas como `static`. No impiden la compilación, pero esas funciones quedan como símbolos globales. Lo recomendado es declararlas `static int __init` y `static void __exit`.

4.4 Paso 3 — Intento de carga en la PC

```
sudo insmod ~/tp4-arm/mimodulo_arm64.ko
sudo dmesg | tail -1
```

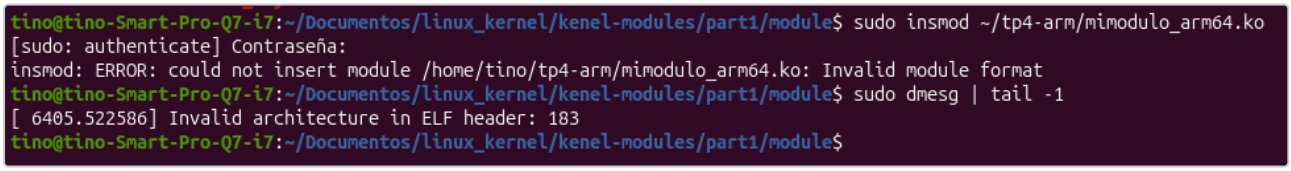


Figura 4 — La PC rechaza el módulo compilado para ARM64.

`insmod` falla con `Invalid module format`, y el log del kernel muestra el motivo: `Invalid architecture in ELF header: 183`. Falla la primera de las verificaciones de la sección 2.3: el campo `e_machine` del módulo vale 183 (`EM_AARCH64`) y el kernel de la PC es x86_64 (62, `EM_X86_64`). El módulo se rechaza en ese punto, antes de llegar a evaluar el `vermagic` o los CRC.

4.5 Paso 4 — Instalación del kernel compilado en la Raspberry Pi

En la PC, se empaquetan la imagen, los módulos y los device trees, y se envían a la Raspberry Pi junto con el módulo:

```
cd ~/rpi/linux
KREL=$(make -s kernelrelease)
make INSTALL_MOD_PATH=../stage INSTALL_MOD_STRIP=1 modules_install
tar -C ../stage/lib/modules -czf ../modulos-tp4.tgz "$KREL"
```

```
cd arch/arm64/boot
tar -czf ~/rpi/boot-tp4.tgz Image dts/broadcom/*.dtb dts/overlays/*.dtbo dts/overlays/README
ping -c 2 raspberrypi.local
scp ~/rpi/modulos-tp4.tgz ~/rpi/boot-tp4.tgz ~/tp4-arm/mimodulo_arm64.ko valendipaolo@raspberryp
i.local:~
```

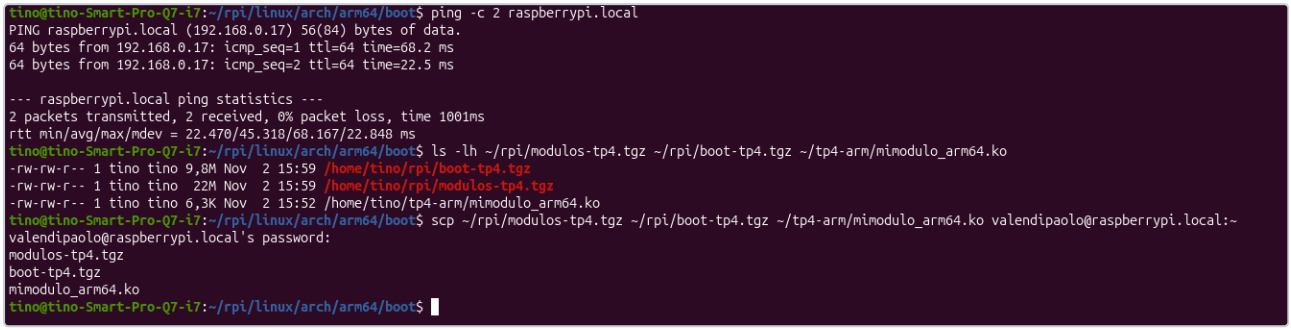


Figura 5 — Envío de los archivos desde la PC a la Raspberry Pi por la red local.

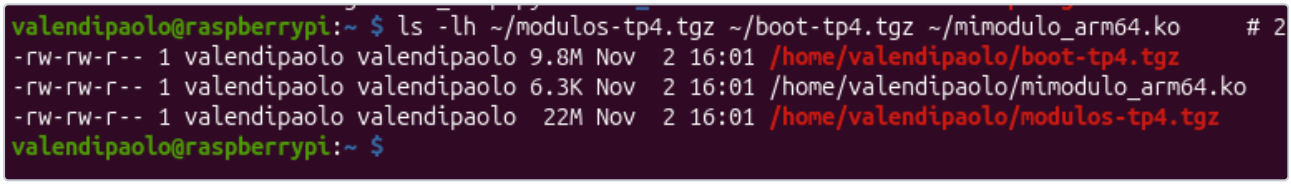


Figura 6 — Archivos recibidos en la Raspberry Pi.

En la Raspberry Pi:

```
sudo cp -a /boot/firmware ~/firmware-backup
sudo tar -C /lib/modules -xzf ~/modulos-tp4.tgz
mkdir -p ~/boot-tp4 && tar -C ~/boot-tp4 -xzf ~/boot-tp4.tgz
sudo cp ~/boot-tp4/Image /boot/firmware/kernel-tp4.img
sudo cp ~/boot-tp4/dts/broadcom/*.dtb /boot/firmware/
sudo cp ~/boot-tp4/dts/overlays/*.dtbo ~/boot-tp4/dts/overlays/README /boot/firmware/overlays/
printf '\n[all]\nkernel=kernel-tp4.img\n' | sudo tee -a /boot/firmware/config.txt
sudo reboot
```

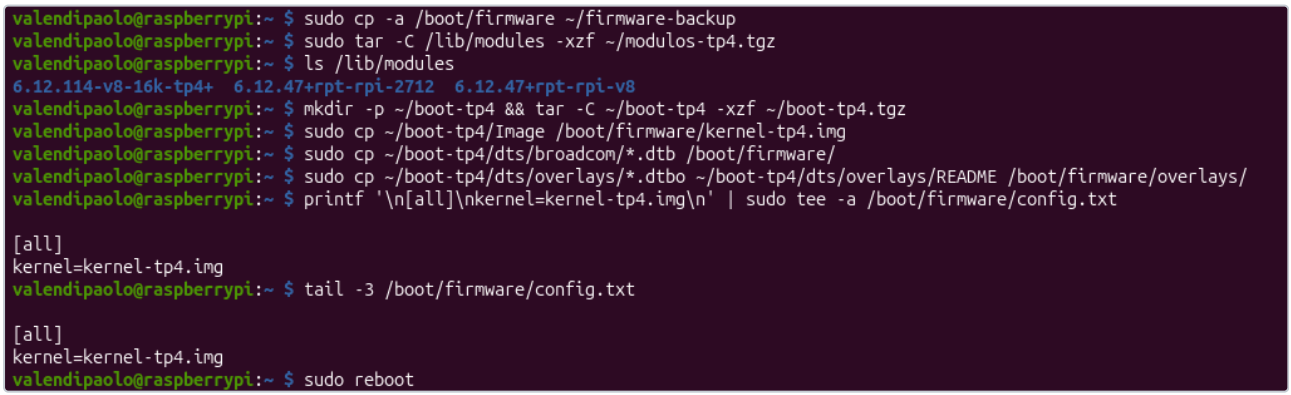


Figura 7 — Instalación del kernel compilado en la Raspberry Pi.

El kernel nuevo se instala con otro nombre (`kernel-tp4.img`) y se selecciona con la opción `kernel=` de `config.txt` . El kernel original queda intacto, así que para volver a él alcanza con borrar esa línea y reiniciar. La sección `[all]` asegura que la línea se aplique sin importar qué sección condicional haya al final del archivo. En `/lib/modules` quedan los módulos de los dos kernels: `6.12.47+rpt-rpi-2712` (original) y `6.12.114-v8-16k-tp4+` (nuevo).

Después del reinicio:

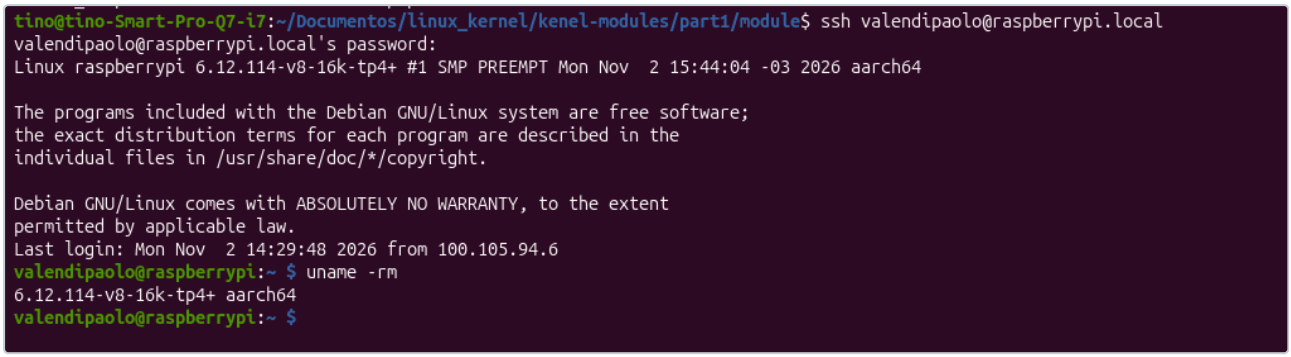


Figura 8 — La Raspberry Pi arranca con el kernel compilado en la PC.

El mensaje de bienvenida de SSH muestra `6.12.114-v8-16k-tp4+ #1 SMP PREEMPT Mon Nov 2 15:44:04 -03 2026`, que coincide con la versión y con la hora de compilación de la imagen en la PC (Figura 2).

4.6 Paso 5 — Carga del módulo en la Raspberry Pi

```
sudo insmod ~/mimodulo_arm64.ko
sudo dmesg | tail -3
lsmod | grep mimodulo
sudo grep mimodulo /proc/modules
sudo rmmod mimodulo
sudo dmesg | tail -1
```

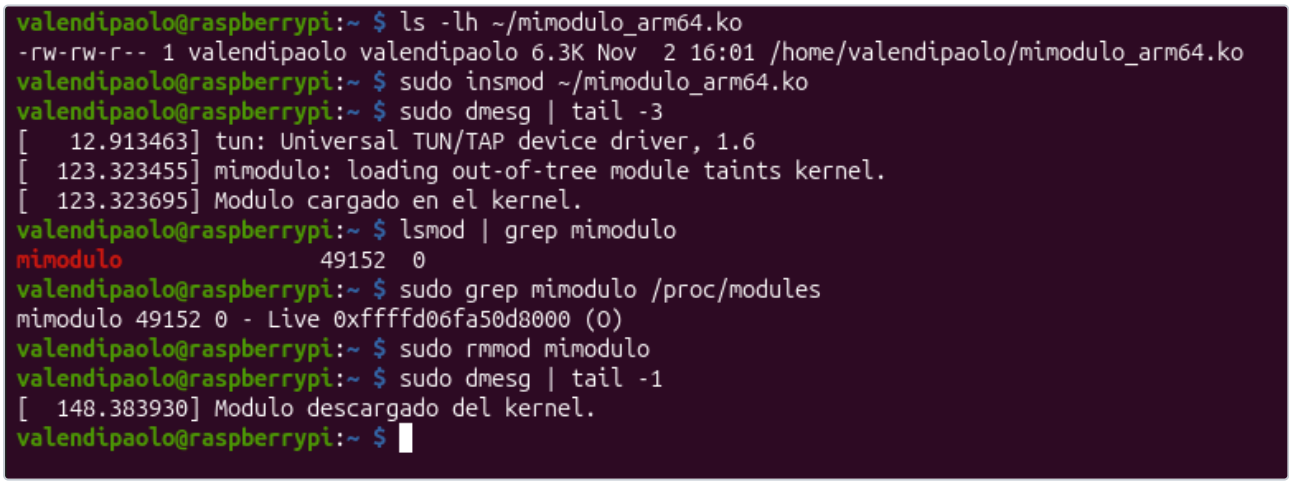


Figura 9 — mimodulo, compilado en la PC, cargado y descargado en la Raspberry Pi.

El mismo archivo que la PC había rechazado se carga sin errores en la Raspberry Pi. El log muestra `mimodulo: loading out-of-tree module taints kernel.` y `Modulo cargado en el kernel.`, y después del `rmmod`, `Modulo descargado del kernel.`

5. Análisis comparativo

	PC (x86_64)	Raspberry Pi 5 (aarch64)
`mimodulo_arm64.ko`	Rechazado: `Invalid architecture in ELF header: 183`	Cargado correctamente
Tamaño en `lsmod`	12288 bytes (módulo x86_64)	49152 bytes
Tamaño de página del kernel	4 KiB	16 KiB
Marcas de *taint*	`(OE)`	`(O)`
Dirección en `/proc/modules`	`0x0000000000000000` (leído sin `sudo`)	`0xffffd06fa50d8000` (leído con `sudo`)

- **Tamaño.** Los dos valores equivalen a exactamente tres páginas de memoria: 12288 = 3 × 4096 y 49152 = 3 × 16384. Es consistente con que el kernel reserve la memoria del módulo en páginas completas. El código fuente es el mismo; lo que cambia es la granularidad de la memoria, porque el kernel de la Raspberry Pi 5 usa páginas de 16 KiB (`CONFIG_ARM64_16K_PAGES=y`).
- **Marcas de *taint*.** En la Raspberry Pi no aparece la **E** (módulo sin firma), porque el kernel compilado con `bcm2712_defconfig` no tiene habilitada la verificación de firmas de módulos (`CONFIG_MODULE_SIG`). El kernel de Ubuntu de la PC sí verifica firmas, y por eso marca los módulos sin firma.
- **Dirección.** El kernel oculta sus direcciones a los usuarios sin privilegios (`kptr_restrict`). Por eso en la PC, leyendo sin `sudo`, aparece `0x0`, mientras que en la Raspberry Pi, con `sudo`, se ve la dirección real donde quedó cargado el módulo.

6. Conclusiones

- A partir del mismo código fuente se obtuvieron dos binarios incompatibles entre sí, uno por arquitectura. Un módulo queda ligado tanto a la arquitectura como al kernel para el que se compiló.
- La PC rechazó el módulo ARM64 en la primera verificación del cargador de módulos, la arquitectura del ELF, sin llegar a evaluar la versión ni los símbolos.
- Para compilar un módulo de forma cruzada no alcanza con los headers del host: hace falta el árbol del kernel del target compilado, con su `Module.symvers`. Por eso el paso más costoso fue compilar el kernel completo de la Raspberry Pi (unos 30 minutos y 1910 módulos), mientras que el módulo en sí se compila en segundos.
- Instalar en la Raspberry Pi el kernel compilado en la PC garantiza que los CRC grabados en el módulo coincidan con los del kernel en ejecución. Con `CONFIG_MODVERSIONS`, son esos CRC, y no la cadena de versión, los que determinan la compatibilidad.
- El procedimiento es reversible: el kernel original de la Raspberry Pi y sus módulos siguen instalados, y se vuelve a ellos editando una línea de `config.txt`.

7. Reproducción

Script	Dónde se ejecuta	Qué hace
`scripts/01_pc_compilar_kernel_rpi.sh`	PC	Instala el compilador cruzado, descarga y compila el kernel de la Raspberry Pi 5 y registra el commit en `rpi-linux-commit.txt`
`scripts/02_pc_crosscompilar_modulo.sh`	PC	Compila `mimodulo` para ARM64, verifica el binario e intenta cargarlo en la PC (debe fallar)
`scripts/03_pc_enviar_a_pi.sh`	PC	Empaqueta el kernel y lo envía a la Raspberry Pi junto con el módulo y los scripts 04 y 05
`scripts/04_pi_instalar_kernel.sh`	Raspberry Pi	Respalda `/boot/firmware`, instala el kernel y lo selecciona en `config.txt`
`scripts/05_pi_probar_modulo.sh`	Raspberry Pi	Carga y descarga el módulo (debe funcionar)

El orden es 01 → 02 → 03 en la PC, y después 04 → `sudo reboot` → 05 en la Raspberry Pi. El destino se puede cambiar con la variable `PI`, por ejemplo `PI=usuario@192.168.0.17 ./scripts/03_pc_enviar_a_pi.sh`. Para reproducir exactamente la misma versión del kernel:

```
COMMIT=$(cut -d' ' -f1 rpi-linux-commit.txt) ./scripts/01_pc_compilar_kernel_rpi.sh
```

Para volver al kernel original de la Raspberry Pi, hay que borrar la línea `kernel=kernel-tp4.img` de `/boot/firmware/config.txt` y reiniciar.

8. Referencias

1. Raspberry Pi Ltd. *The Linux kernel — Build the kernel; Cross-compile the kernel*. https://www.raspberrypi.com/documentation/computers/linux_kernel.html
2. P. J. Salzman, M. Burian, O. Pomerantz, B. Mottram, J. Huang. *The Linux Kernel Module Programming Guide*. https://sysprog21.github.io/lkmpg/
3. Repositorio del kernel de Raspberry Pi, rama `rpi-6.12.y`. https://github.com/raspberrypi/linux
4. Código fuente del kernel Linux 6.12: `kernel/module/main.c`, `kernel/module/version.c`, `include/uapi/linux/elf-em.h`.
5. Cátedra de Sistemas de Computación (FCEFyN, UNC). Presentación *Linux Kernel Module Programming I* y repositorio *kenel-modules*. https://gitlab.com/sistemas-de-computacion-unc/kenel-modules